

Smart Lockbox System

Consolidated Review 1 + Review 2 + Review 3 Report

Name: - Venkata Sriram Topalli

Reg No: - 23BCE0441

1. Executive Summary

The Smart Lockbox project is an IoT-based access control system that combines an ESP32 microcontroller, Firebase Realtime Database, a web application, an ultrasonic sensor, a servo motor, and an active buzzer. The system forms a closed-loop cyber-physical workflow: the web interface writes commands to Firebase, the ESP32 reads them, executes the physical action, and writes sensor-derived status back to the database.

The consolidated report presents the project as a complete solution across architecture, data handling, algorithm design, execution layers, user interface, real-time synchronization, calibration, test cases, performance metrics, prototype validation, and comparison with commercial alternatives.

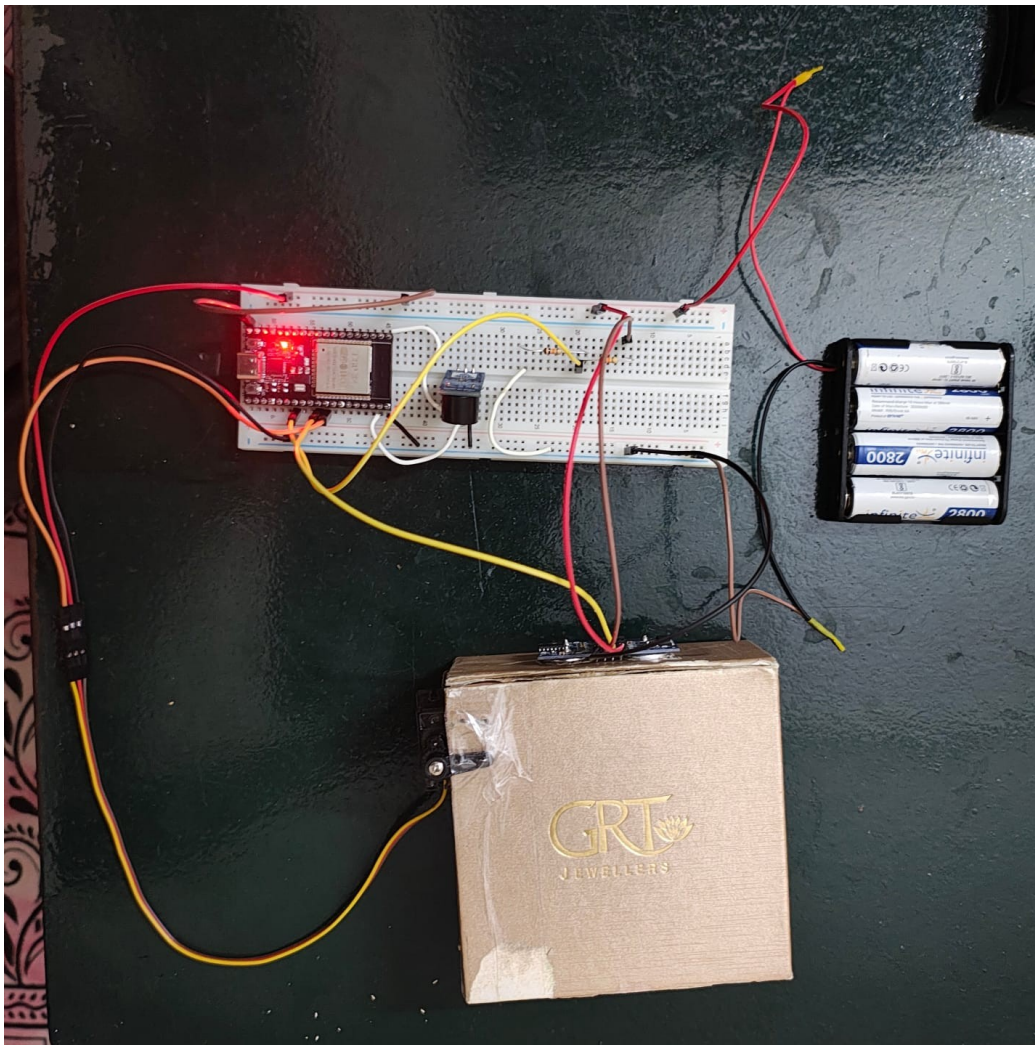


Figure 1: Complete smart lockbox prototype assembled for demonstration

2. Project Overview and Scope

The Smart Lockbox System is an Internet of Things (IoT) based access-control solution designed to securely store and manage physical keys. The system integrates embedded hardware, cloud infrastructure, and a web-based user interface to enable controlled and monitored access to a physical asset.

The ESP32 functions as the main controller and communicates with Firebase Realtime Database to synchronize the hardware state with the web application. This enables the system to act as a cyber-physical loop in which user actions become database events, database events trigger physical actions, and physical feedback is reported back to the user interface.

Scope Type	Key Points
Functional scope	Remote lock/unlock, key presence detection, buzzer alerts, role-based workflows
Technical scope	ESP32 control, Firebase communication, Wi-Fi connectivity, event-driven firmware
Performance scope	Fast response, stable sensing, reliable communication, efficient resource use
Application scope	Real-estate access, rental services, shared facilities, secure storage
Limitations	Requires Wi-Fi, sensor calibration matters, power optimization can be improved

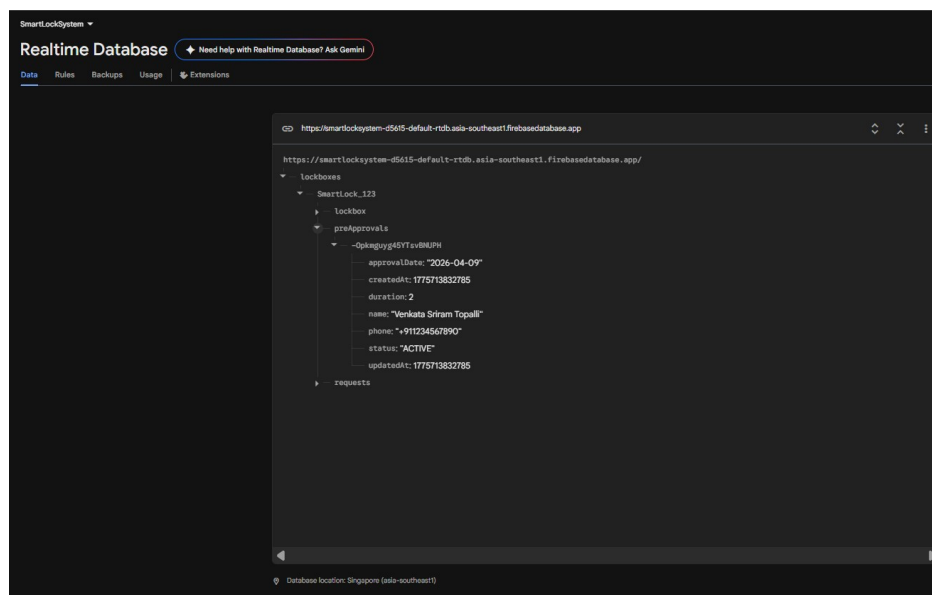
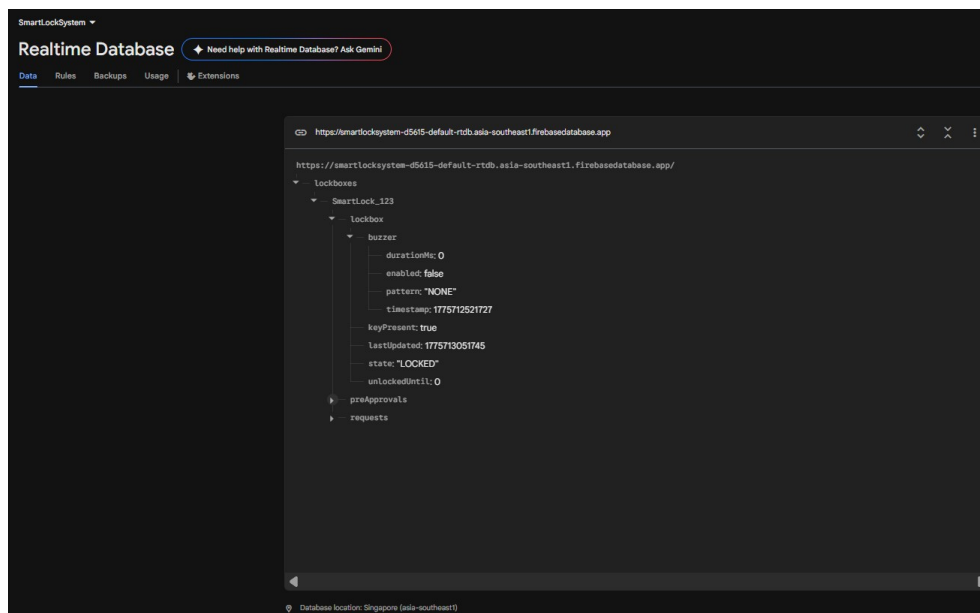
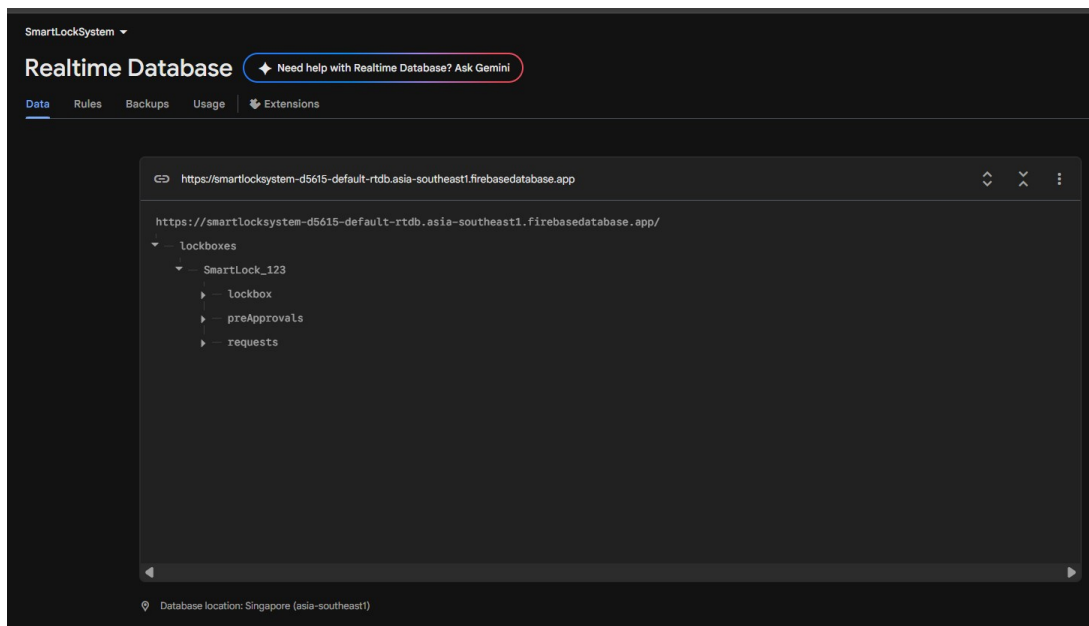
3. Dataset and Database

3.1 Dataset

The project does not depend on a fixed offline dataset. Instead, it generates a real-time operational dataset from live user interaction, sensor readings, and internal system events. The dataset is therefore dynamic, event-driven, and time-dependent.

Data Type	Examples	Source	Purpose
Control data	LOCKED, UNLOCKED, WARNING_1_MIN	Web application	Controls the lockbox
Sensor data	Distance, key present / absent	HC-SR04 ultrasonic sensor	Detects the key state
System state data	Lock state, operating mode	ESP32 logic	Maintains system status
Event data	Unlock, return, alert events	ESP32 and web app	Supports logging and validation

Firebase Path	Direction	Meaning	Example Value
/state	Read	Lock command from web app	LOCKED / UNLOCKED
/buzzer/pattern	Read	Alert trigger pattern	WARNING_1_MIN
/keyPresent	Write	Ultrasonic key status	true / false
/lockbox	Read/Write	Main device state container	JSON object
/requests	Website-side	Request history records	JSON object



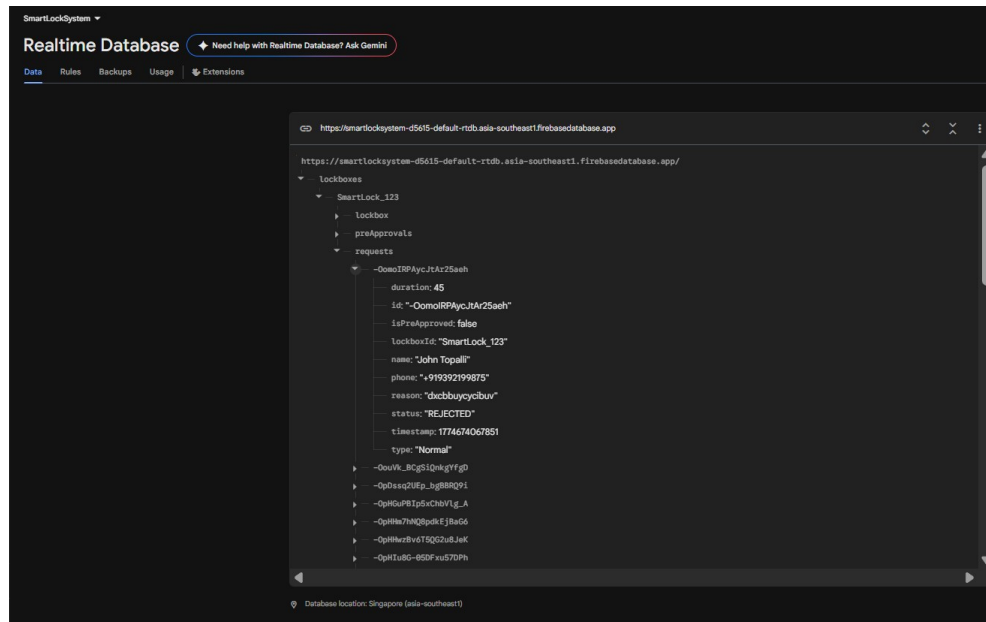


Figure 2: Different nodes in Firebase Realtime Database

3.2 Database

The system uses Firebase Realtime Database, a cloud-hosted NoSQL database that stores information in a JSON tree format. Its real-time synchronization and low-latency update model make it suitable for a reactive embedded system.

- Real-time synchronization
- Low-latency updates
- Cloud-based access
- Automatic propagation of changes

The database is organized hierarchically as follows:

```
lockboxes/
  SmartLock_123/
    lockbox/
      state: "LOCKED" / "UNLOCKED"
      keyPresent: true / false
      buzzer/
        pattern: "NONE" / "WARNING_1_MIN" / "KEY_NOT_RETURNED"
```

The ESP32 reads the state and buzzer pattern from Firebase, while the sensor-derived key status is written back to the same database so the website can display the live physical state.

4. Algorithm Description

The Smart Lockbox algorithm is event-driven, state-based, and divided between communication and hardware execution. The ESP32 listens to Firebase updates, converts relevant changes into command flags, and then the hardware loop performs the physical actions.

Task	Responsibility	Output
Core 0 / Network task	Reads Firebase stream and converts updates into flags	cmdOpenBox, cmdCloseBox, cmdStartBuzzer
Core 1 / Hardware task	Executes servo, buzzer, and sensor logic	Physical lock/unlock, alerts, key detection

The algorithm operates in four main stages:

1. Connect – establish Wi-Fi and Firebase connection
2. Listen – monitor Firebase for incoming updates
3. Act – execute servo, buzzer, and sensor-related actions
4. Report – upload key-state updates back to Firebase

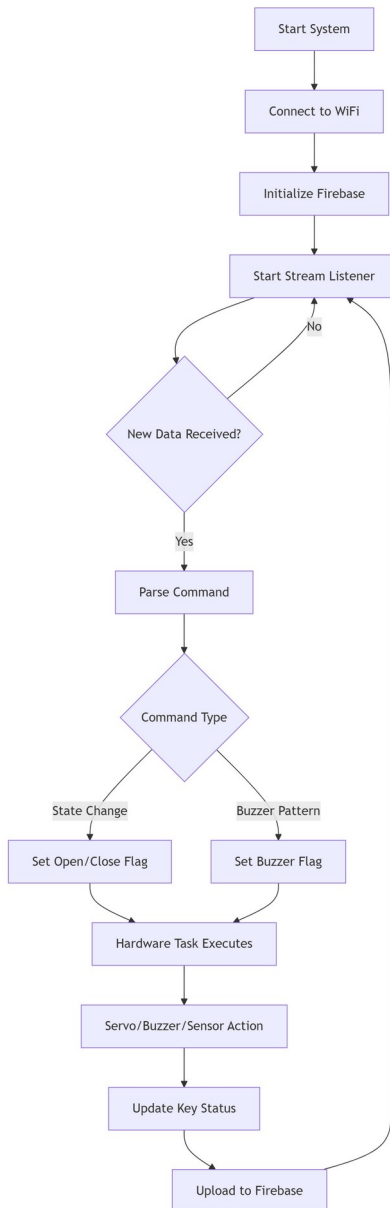


Figure 3: High-level algorithm flow

Command processing is handled by the network task, which listens to Firebase stream updates.

- Read updated data from Firebase stream
- Identify the data path (/state, /buzzer/pattern)
- Convert data into internal command flags
- Avoid duplicate triggers using lastBuzzerPattern

Servo, buzzer, and sensor logic are executed locally on the ESP32.

- Servo moves only when the lock state changes
- Buzzer runs for a fixed duration with non-blocking toggling
- Ultrasonic sensor is read only during unlocked state
- Key changes are accepted only after a 1-second stability window

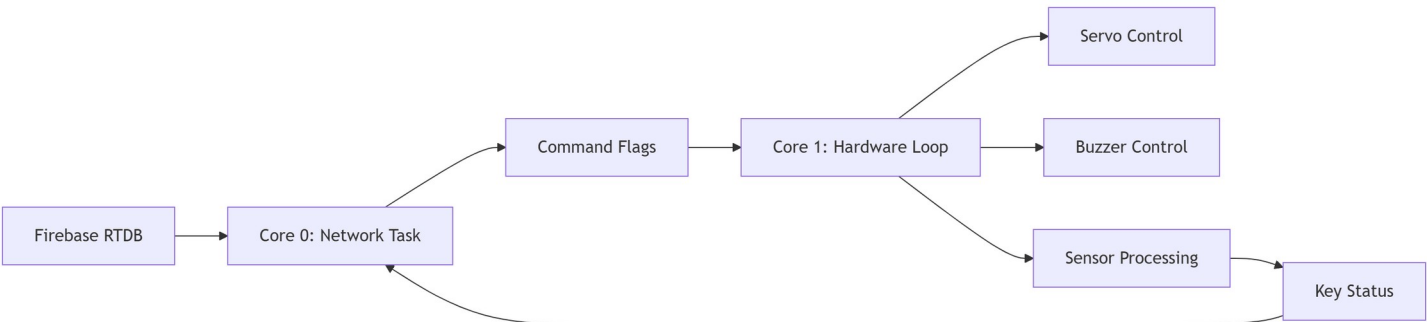


Figure 4: Dual-core execution diagram

The full source code is stored in the GitHub repository. In this report, the algorithmic behavior is emphasized rather than reproducing the entire code listing.

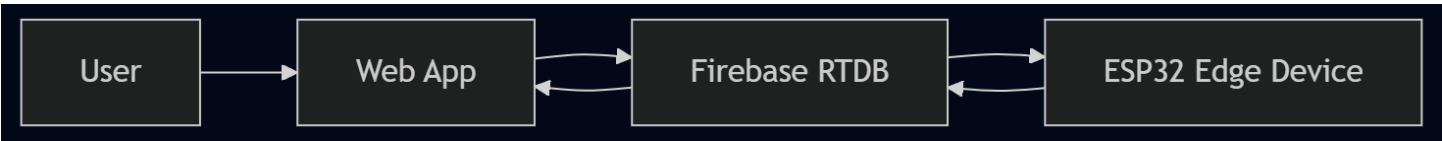
5. Execution of the Algorithm: Edge, Fog, and Cloud

The Smart Lockbox system executes across three layers. The cloud layer manages the web interface and access workflow, the fog layer acts as the synchronization bridge through Firebase Realtime Database, and the edge layer performs the physical operations on the ESP32 and peripherals.

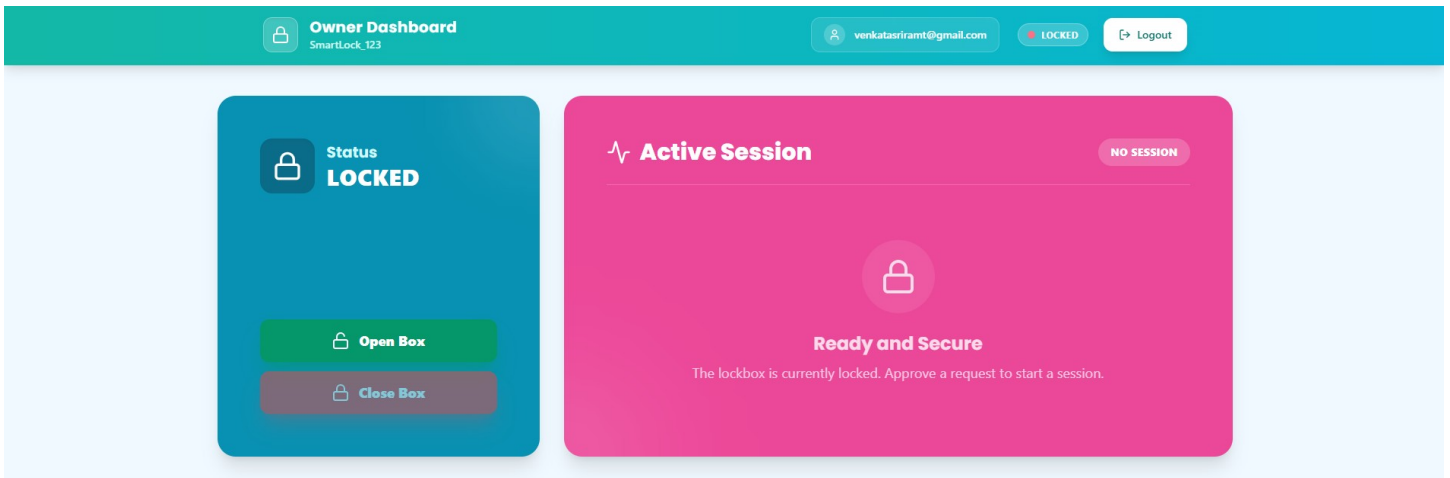
Layer	Where it runs	What it does	Why it matters
Cloud	Web application and backend logic	Login, dashboard updates, logs, access requests	Enables remote control and monitoring
Fog	Firebase Realtime Database	Shared state storage between web app and ESP32	Synchronizes commands and sensor feedback
Edge	ESP32 and peripherals	Servo motion, buzzer timing, ultrasonic sensing	Provides immediate physical response

Execution description:

- Cloud execution handles user interaction, authentication, and status display.
- Fog execution appears in Firebase, which temporarily stores commands and sensor feedback.
- Edge execution happens in the ESP32 sketch, where time-sensitive physical actions are performed locally.



[Figure 5: System-level flow from user to web app, Firebase RTDB, and ESP32 edge device]



[Figure 6: Owner Dashboard showing the active session and the open and close buttons]

```

Serial Monitor X Output
Message (Enter to send message to 'ESP32 Dev Module' on 'COM3') New Line 115200 baud
22:42:48.933 -> Connecting WiFi.....
22:42:56.418 -> WiFi connected
22:42:58.306 -> System Ready - Cores Split & Collision Protection Active
22:42:58.538 -> NETWORK: Unlock command received
22:42:59.094 -> HARDWARE: Lockbox OPENED & Servo Detached
22:43:00.133 -> HARDWARE: Stable Key State Detected -> MISSING
22:43:01.004 -> NETWORK: Safe Upload Success -> KEY MISSING
22:43:20.538 -> NETWORK: Buzzer command received -> WARNING_1_MIN
22:43:20.538 -> HARDWARE: Buzzer Started
22:43:30.553 -> HARDWARE: Buzzer Stopped

```

Figure 7: Logs from ESP32 showing booting (connecting to wifi) and working off servo, ultrasonic sensor and buzzer

6. User Interface

The user interface is divided into owner and guest workflows so that each user sees only the actions relevant to them. The design focuses on clarity, real-time feedback, and simple state transitions.

UI Module	Main Purpose	Key Functions
Owner Dashboard	Property management and supervision	Manage properties, grant access, view request history, monitor alerts
Guest Dashboard	Visit execution and access handling	View assigned property, unlock box, track visit status, complete visit
Status Panel	Live system feedback	Locked, unlocked, key present, key missing, buzzer active

Owner interface:

- Manage properties and lockbox entries
- Grant access for a specific user and time
- View request history and access logs
- Monitor buzzer alerts and lock state in real time

Guest interface:

- Login and verification
- View assigned property
- Unlock the lockbox

- Check visit status
- Finish visits after returning the key

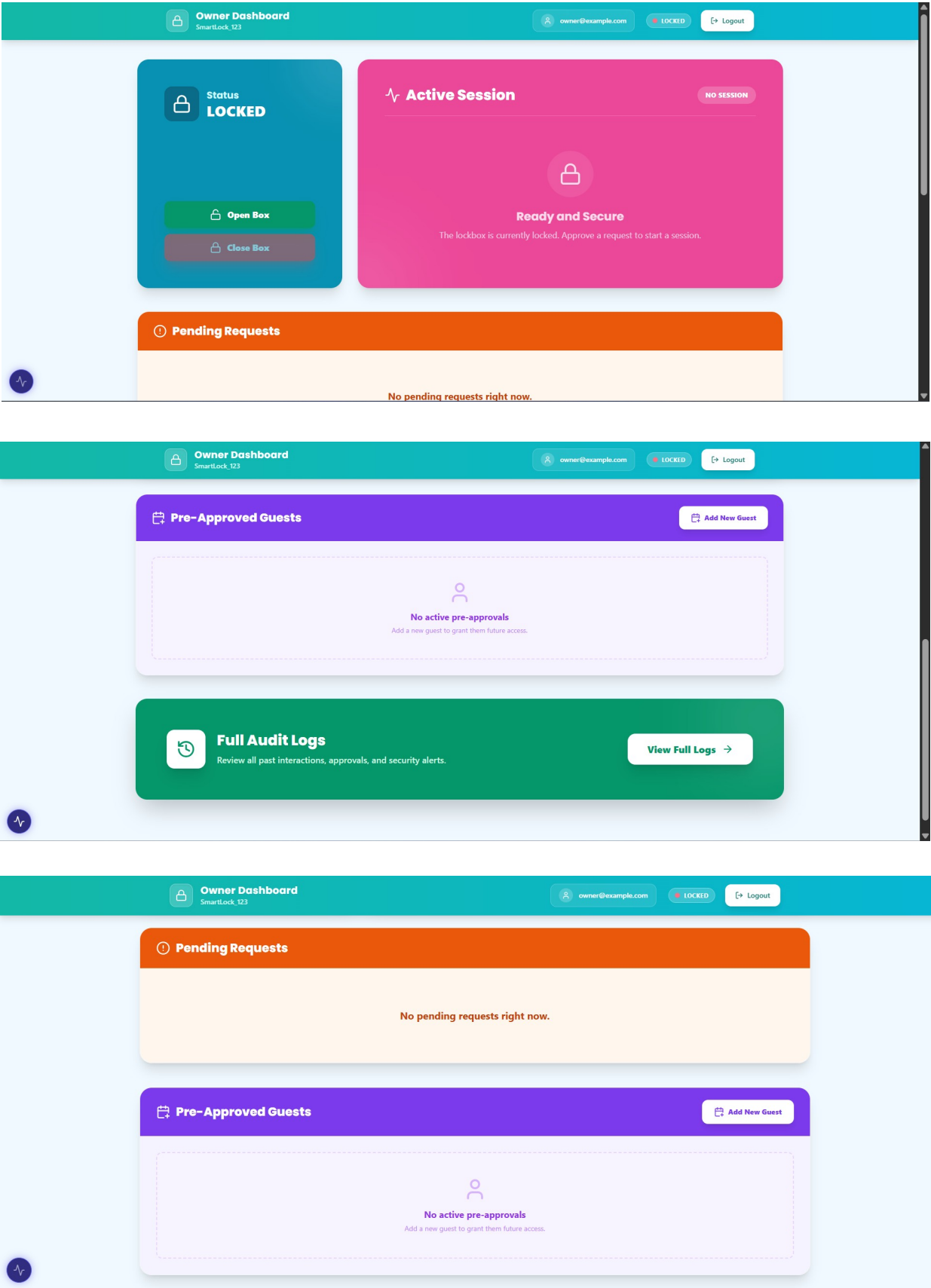


Figure 8: Owner Dashboard

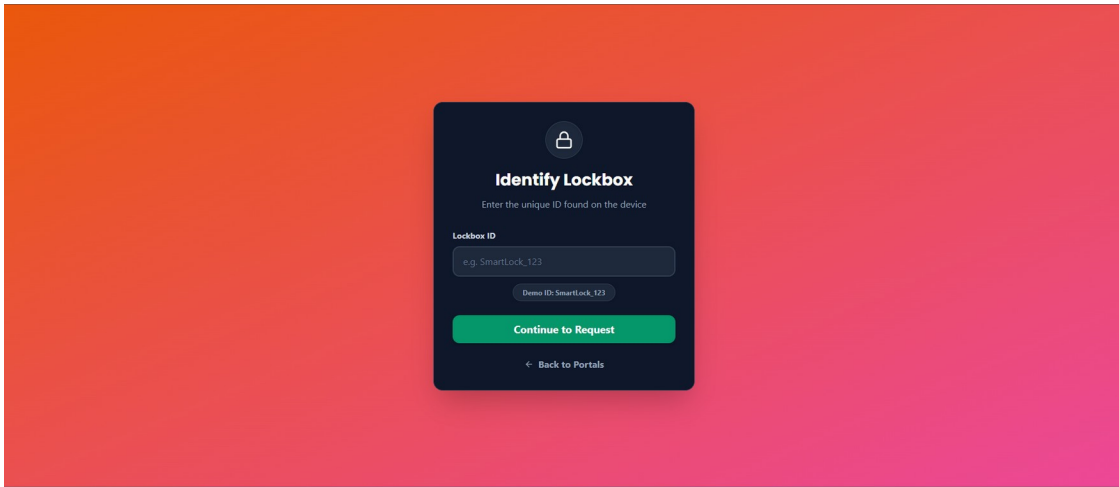
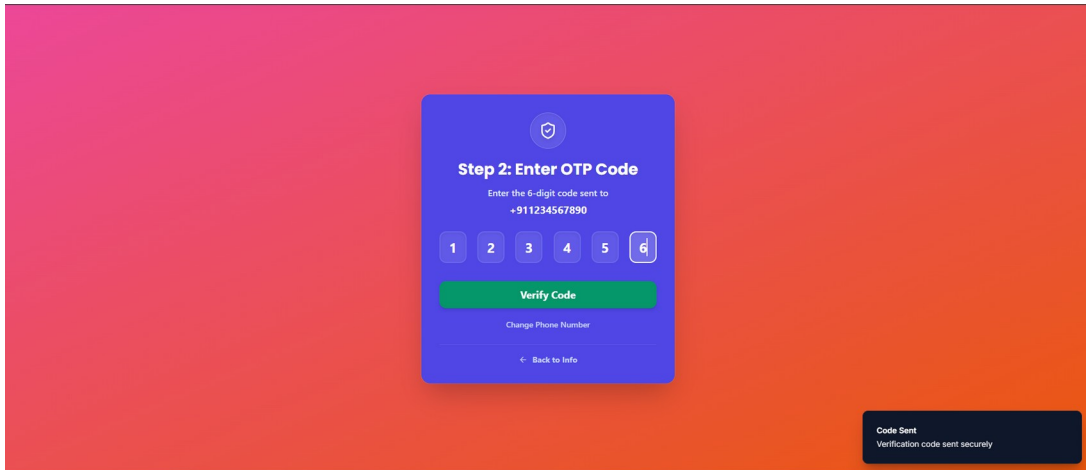



Figure 9: Guest sending a normal request workflow (entering lockbox id, filling guest details, and OTP verification)

7. Real-Time Data Handling

The Smart Lockbox system is a real-time reactive system. Instead of repeatedly polling the database, the ESP32 uses the Firebase stream API to listen for changes as they occur. This reduces delay, lowers network overhead, and improves responsiveness.

Event	Source	Action Taken	Real-Time Benefit
Lock state change	Firebase	Servo opens or closes the box	Immediate access response

Buzzer pattern update	Firebase	Starts timed alert sequence	Fast warning delivery
Key presence change	Ultrasonic sensor	Uploads updated key status	Accurate physical synchronization
Stable distance detected	ESP32 logic	Confirms key present/missing	Reduces false readings

The real-time data flow is straightforward:

- The web app updates Firebase with a new command.
- Firebase stream immediately detects the change.
- The ESP32 executes the corresponding physical action.
- Sensor results are written back to Firebase and shown in the UI.

The system uses four important real-time mechanisms:

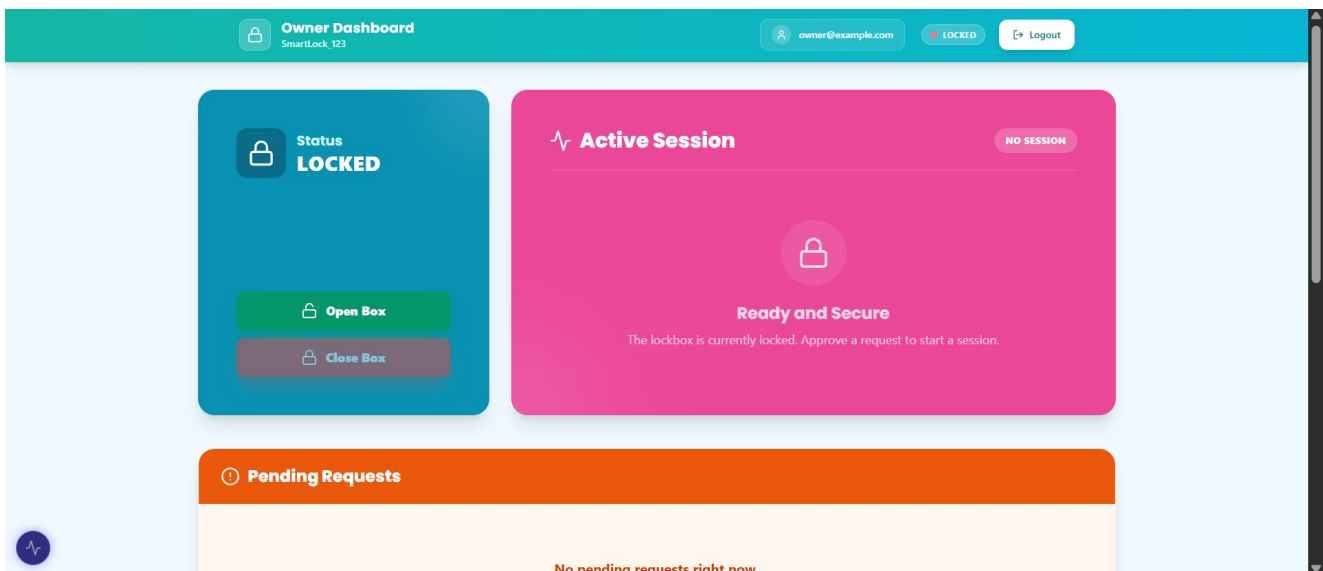
1. Stream-based listening
2. Change-based uploading of keyPresent
3. Pattern-based buzzer control
4. Stable sensor validation

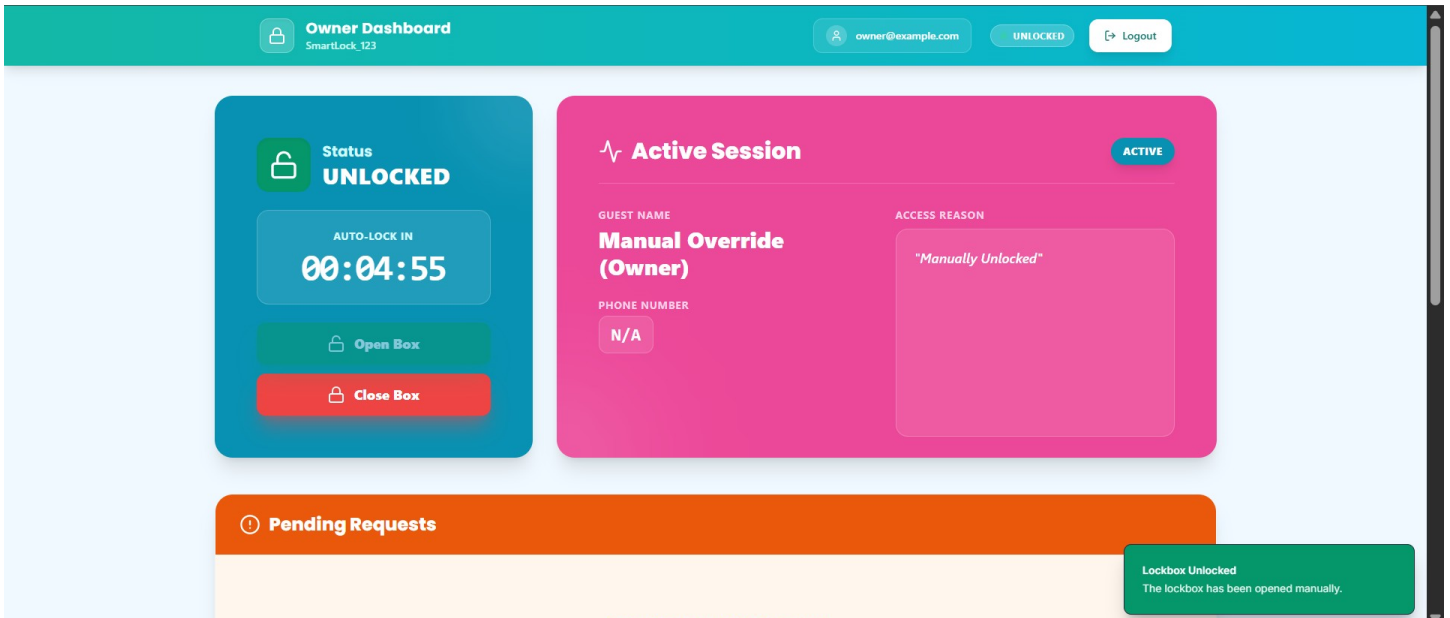
```

Serial Monitor x Output
Message (Enter to send message to 'ESP32 Dev Module' on 'COM3') New Line 115200 baud
22:42:48.933 -> Connecting WiFi.....
22:42:56.418 -> WiFi connected
22:42:58.306 -> System Ready - Cores Split & Collision Protection Active
22:42:58.538 -> NETWORK: Unlock command received
22:42:59.094 -> HARDWARE: Lockbox OPENED & Servo Detached
22:43:00.133 -> HARDWARE: Stable Key State Detected -> MISSING
22:43:01.004 -> NETWORK: Safe Upload Success -> KEY MISSING
22:43:20.538 -> NETWORK: Buzzer command received -> WARNING_1_MIN
22:43:20.538 -> HARDWARE: Buzzer Started
22:43:30.553 -> HARDWARE: Buzzer Stopped

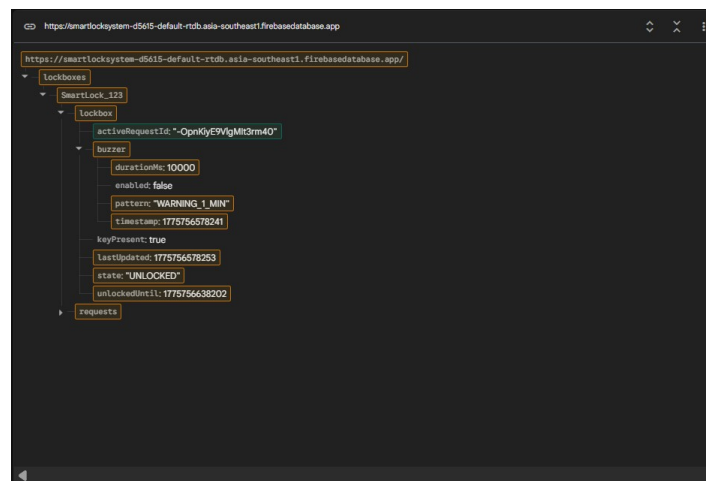
```

[Figure 10: Firebase stream update received by ESP32 in Serial Monitor]





[Figure 11: Lock state changing from LOCKED to UNLOCKED in the web UI]



[Figure 12: Firebase database showing updated state, keyPresent, and buzzer/pattern]

8. Sensor Calibration and Validation

The Smart Lockbox uses an HC-SR04 ultrasonic sensor to determine whether the key is present inside the box. Because ultrasonic readings vary with mounting position, object shape, and reflection angle, the sensor is calibrated using the empty-box condition as the reference threshold.

In the current firmware, the empty range is set between 7.0 cm and 8.5 cm. Readings inside this range are treated as key missing; readings outside are treated as key present. A stable reading is required before a new state is accepted and uploaded to Firebase.

Stage	Method	Purpose	Outcome
Empty-box baseline	Measure distance without the key	Find the normal vacant-slot range	Set EMPTY_LOW and EMPTY_HIGH
Occupied-box baseline	Place the key in its normal position	Confirm presence detection	Validate true-positive reading

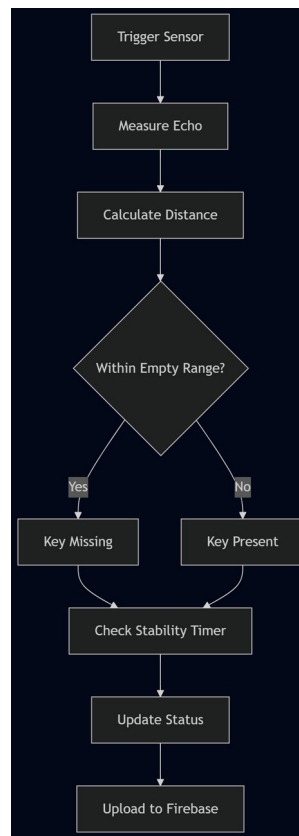
Stability test	Repeat readings over time	Check consistency	Use debounce and 1-second confirmation window
Functional validation	Remove and reinsert the key	Verify state transitions	Update keyPresent only after stable change

Calibration logic:

- If the measured distance is inside the empty range, the slot is considered empty.
- If the measured distance is outside the empty range, the slot is considered occupied.
- A stable reading is required before changing the database value.

Validation method:

1. Keep the box empty and observe the baseline readings.
2. Place the key back in the holder and verify the occupied reading.
3. Remove the key again and confirm that the database changes only after stability.



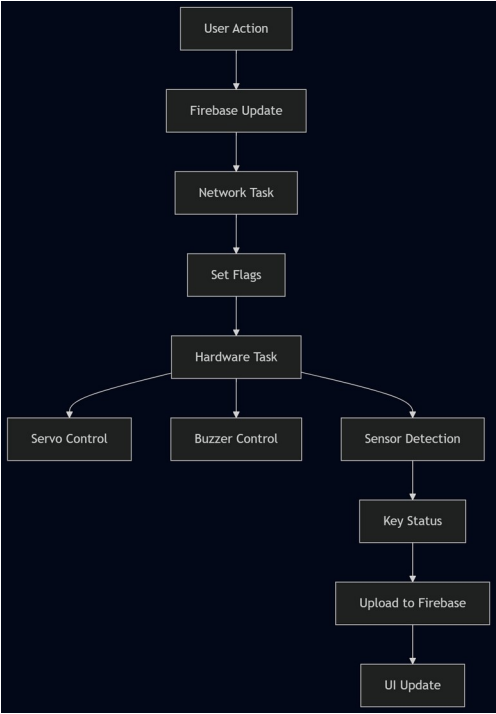
[Figure 13: Sensor validation flow]

Condition	Expected Reading	System Interpretation	Database Action
Slot empty	7.0–8.5 cm	Key missing	Upload keyPresent = false
Key in place	Outside empty range	Key present	Upload keyPresent = true
Reading fluctuates	Varies slightly	Ignore until stable	No immediate update
State change confirmed	Stable for 1 second	Valid transition	Write new value to Firebase

9. Test Cases

The following test cases cover the main functional behavior of the prototype and validate the lock/unlock workflow, buzzer behavior, and key detection logic.

Test Case	Input	Expected Result	Observed in Code	Status
TC1	UNLOCKED command	Servo opens, system becomes active	cmdOpenBox flag is set	Pass
TC2	LOCKED command	Servo closes and detaches	cmdCloseBox flag is set	Pass
TC3	WARNING_1_MIN pattern	Buzzer beeps for 10 seconds	Timed beep sequence starts	Pass
TC4	Key removed while unlocked	keyPresent becomes false	Distance crosses empty threshold	Pass
TC5	Key returned after absence	keyPresent becomes true	Stable reading uploads after debounce	Pass
TC6	Repeated same alert	Alert should not restart	lastBuzzerPattern blocks duplicate triggers	Pass



[Figure 14: Functional flow for command, sensor, and buzzer validation]

10. Metrics and Evaluation

The system is evaluated using execution time, response time, memory usage, power consumption, communication reliability, and sensor accuracy. Because the project is embedded and real-time in nature, response time and detection accuracy are the most important metrics.

Metric	Measurement Method	Formula / Basis	Why it Matters	Current Observation
Execution time	Measure delay from command reception to servo action	Command-to-actuation latency	Shows time efficiency	Reduced by flag-based execution and short loop delay
Space efficiency	Estimate RAM and flash usage	Code size and variable footprint	Shows resource efficiency	Small state set; library-based abstraction
Power efficiency	Measure current in	Idle current vs active	Important for battery	Servo detached after

	idle and active mode	current	life	motion; sleep optimization possible
Response time	Time from Firebase update to hardware response	Firebase update → physical action delay	Shows real-time behaviour	Stream-based updates reduce delay
Communication efficiency	Count successful, lost, or repeated updates	PDR, packet loss, duplicate avoidance	Shows network reliability	Retry and state comparison reduce repeats
Sensor accuracy	Compare ultrasonic result with manual key check	Correct detections / total tests	Ensures reliable key detection	Calibration range already defined in firmware

Estimated performance ranges (prototype-level observations):

Metric	Typical / Estimated Value
State-change delay	~100–300 ms
Sensor distance band	7.0–8.5 cm
Alert duration	~10 seconds
Update frequency	~100–150 ms cycle

11. Qualitative and Quantitative Measures

Qualitative measures focus on usability, clarity, and real-world behavior.

Aspect	Description	Observation
User clarity	UI clearly shows lock and key status	Users can easily understand system state
Operational simplicity	Minimal steps to unlock and complete visit	Smooth owner and guest workflows
Physical behavior	Servo and sensor reflect real actions	Lockbox behaves predictably
Alert visibility	Buzzer and UI alerts are noticeable	Alerts are clear and timely

Quantitative measures focus on measurable system performance.

Metric	What is Measured	Typical / Estimated Value
State-change delay	UI command → hardware action	~100–300 ms
Sensor distance band	Empty range calibration	7.0–8.5 cm
Alert duration	Buzzer active time	~10 seconds
Update frequency	Sensor and database updates	~100–150 ms cycle

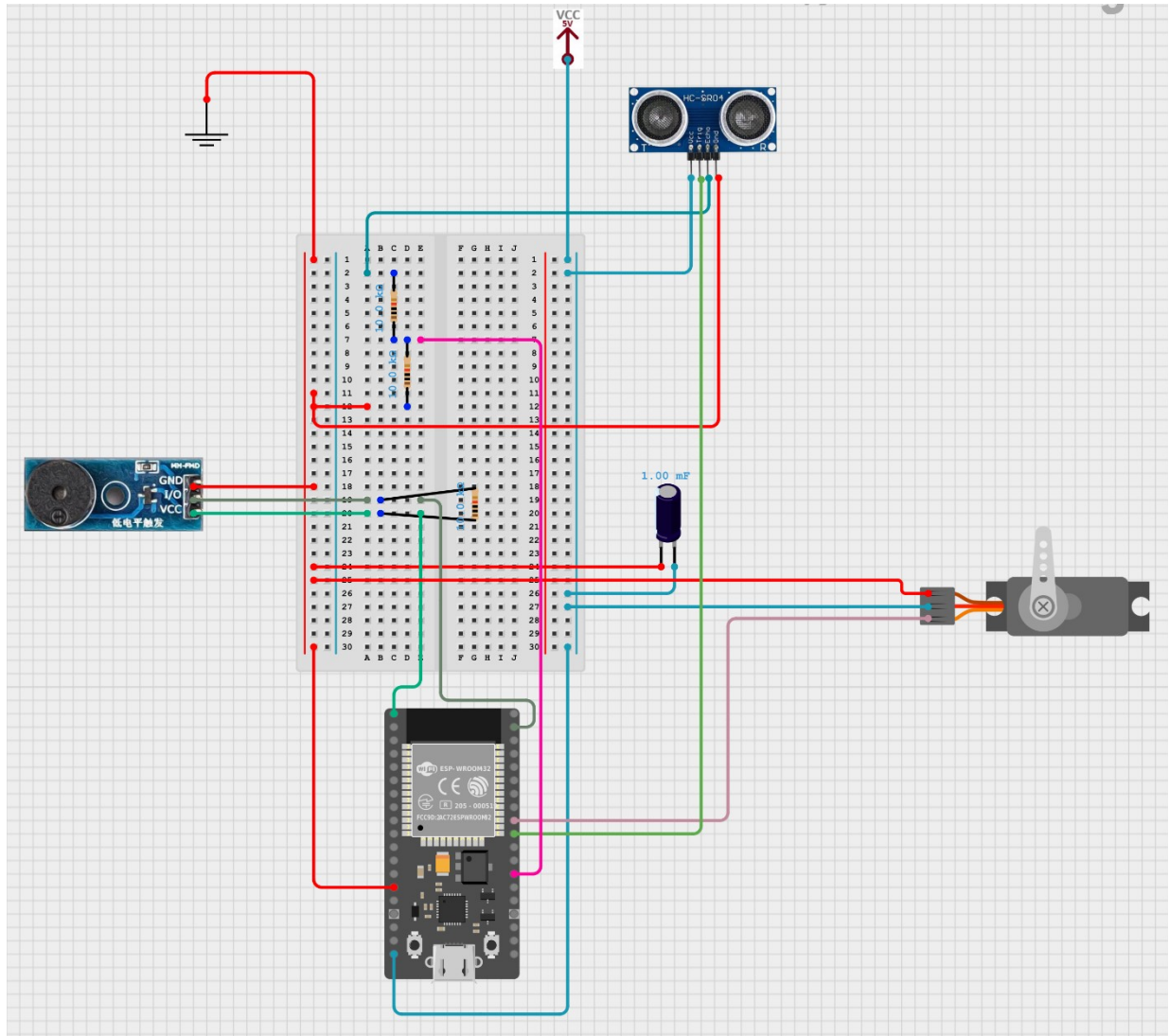
12. Hardware Prototype and Working Model

The hardware prototype is organized as a closed-loop embedded system made up of four main subsystems: the controller, sensor, actuator, and alert unit. Together, these parts allow the Smart Lockbox to receive remote commands, physically unlock or lock the box, detect whether the key has been returned, and notify the user through sound and database updates.

Subsystem	Role	Working Behavior
ESP32 Controller	Main control unit	Connects to Wi-Fi, reads Firebase, executes logic
Servo Motor	Lock actuator	Opens and closes the box on command
Ultrasonic Sensor	Key detection unit	Detects key presence only during unlocked state
Buzzer	Alert unit	Produces timed warning pattern
Firebase Database	Communication layer	Stores lock state, key status, and buzzer commands

Working model sequence:

1. ESP32 boots and establishes Wi-Fi connection.
2. Firebase is initialized and the stream listener starts.
3. Servo is initialized in the locked position.
4. When an UNLOCKED command arrives, the servo opens the box.
5. During the unlocked period, the ultrasonic sensor monitors key presence.
6. If the key is removed or returned, the system waits for stability before updating Firebase.
7. If an alert pattern is received, the buzzer sounds for the defined duration.
8. After the action completes, the system returns to monitoring mode.



[Figure 15: Full circuit diagram showing ESP32, servo, ultrasonic sensor, and buzzer]

12.1 Wiring Details

12.1.1 Electrolytic Capacitor

- **Negative (-) Pin:** Connected to GND (common ground).
- **Positive (+) Pin:** Connected to VCC 5V and ESP32 5V pin.

12.1.2 Buzzer Module

- **GND Pin:** Connected to GND (common ground).
- **Vcc Pin:** Connected to ESP32 3.3V pin.
- **I/O Pin:** Connected to ESP32 GPIO 23 (VSPI MOSI) through a 10k Ohm resistor.

12.1.3 Servo

- **GND Pin:** Connected to GND (common ground).
- **VCC Pin:** Connected to VCC 5V.
- **PWM Pin:** Connected to ESP32 GPIO 18 (VSPI CLK).

12.1.4 Sensor - Ultrasonic HC-SR04

- **Trig Pin:** Connected to ESP32 GPIO 5 (VSPI CS).
- **Echo Pin:** Connected to ESP32 GPIO 4 through a 10k Ohm resistor.
- **GND Pin:** Connected to GND (common ground).
- **VCC Pin:** Connected to VCC 5V.

12.1.5 VCC 5V

- **VCC Pin:** Provides 5V power to the circuit.

12.1.6 ESP32 38-PIN

- **GND Pin:** Connected to GND (common ground).
- **5V Pin:** Connected to VCC 5V.
- **3.3V Pin:** Connected to Buzzer Module Vcc pin.
- **GPIO 23 (VSPI MOSI):** Connected to Buzzer Module I/O pin through a 10k Ohm resistor.
- **GPIO 18 (VSPI CLK):** Connected to Servo PWM pin.
- **GPIO 5 (VSPI CS):** Connected to Sensor Trig pin.
- **GPIO 4:** Connected to Sensor Echo pin through a 10k Ohm resistor.

12.1.7 GND

- **GND Pin:** Common ground connection for all components.

12.1.8 Resistors (10k Ohms)

- **Resistor 1:** Connected between Buzzer Module I/O pin and ESP32 GPIO 23.
- **Resistor 2:** Connected between Sensor Echo pin and ESP32 GPIO 4.
- **Resistor 3:** Used in the ground network.

13. Comparative Analysis with Existing Solutions

The proposed lockbox can be compared against commercial alternatives in terms of cost, connectivity, offline behavior, key monitoring, auditability, and customization. The comparison highlights the main differentiator of the student-built prototype: low cost and full control over both code and user interface.

Aspect	Your Smart Lockbox (ESP32 + Firebase)	Master Lock Smart Lockbox	Igloohome Smart Lockbox
Cost	Medium-cost prototype	High / premium branded product	High / premium smart solution
Connectivity	Wi-Fi + Firebase real-time sync	Bluetooth + proprietary app	Bluetooth + offline PIN system
Cloud integration	Open Firebase backend	Closed ecosystem	Partial cloud + offline hybrid
Offline capability	Limited / future scope	Limited	Strong via algoPIN
Key monitoring	Ultrasonic-based key detection	Not typically available	Not typically available
Audit logs	Possible via Firebase	Limited to vendor ecosystem	Advanced time-based access logs
Customization	Fully customizable	Very limited	Limited to vendor-defined features
Main advantage	Low cost + full control	Reliable hardware	Works without internet

Main limitation	Prototype-level robustness	Expensive and less flexible	Expensive and closed system
-----------------	----------------------------	-----------------------------	-----------------------------

The comparison indicates that the prototype fills an important gap between affordability and functionality. Commercial solutions are polished, but the student-built system provides stronger customizability and the unique feature of key-return monitoring.

Cost: -

ESP32 WROOM 32 - ₹634.00
 4 Ni-MH Batteries with charger - ₹855.00
 3 10 kΩ resistors - ₹10.00
 Breadboard - ₹59.00
 Wires and jumper cables - ₹ 107.00
 Ultrasonic Sensor - ₹ 75.00
 Battery pack holder - ₹ 40.00
 TowerPro MG90S Micro Servo (180° rotation) - ₹ 150.00
 ADIY Active Buzzer Module - ₹ 20.00
 1000μF Capacitor - ₹ 20.00
 Website Development - ₹ 9490.98
 Firebase - ₹ 20.00

Total Cost: - ₹ 11480.98

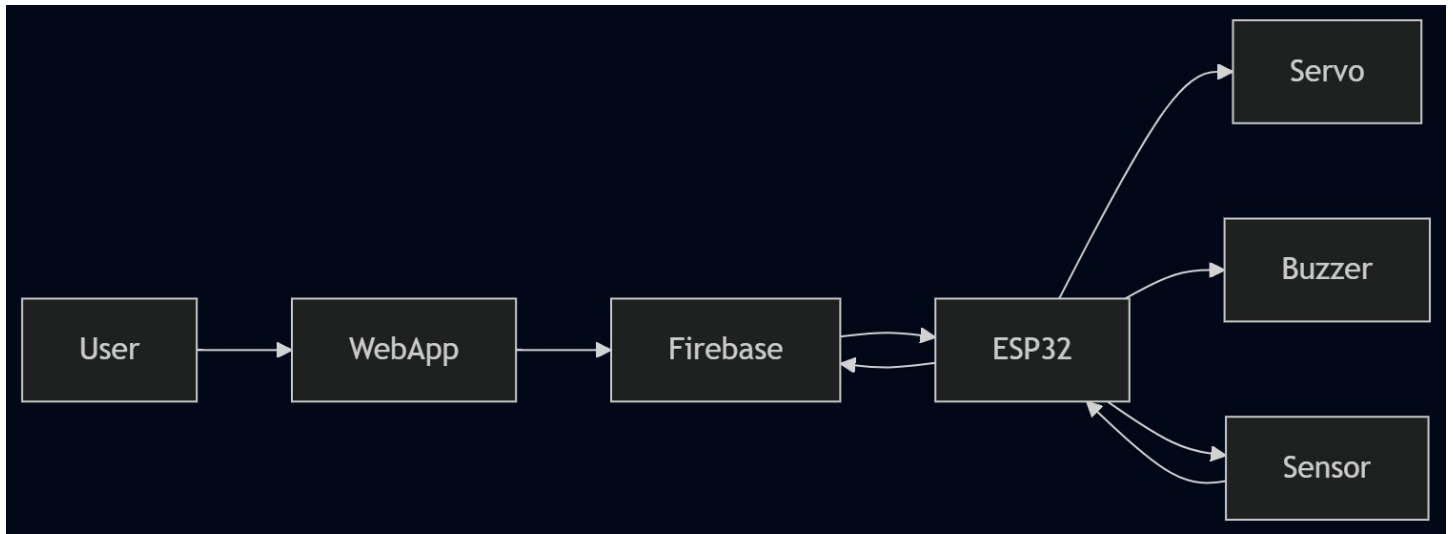
14. Conclusion

The Smart Lockbox project demonstrates a complete embedded-to-cloud control loop. The ESP32, ultrasonic sensor, servo motor, buzzer, Wi-Fi link, and Firebase database work together to deliver a practical access-management system. The most important strengths of the implementation are the real-time synchronization architecture, the state-based firmware, and the key-return monitoring concept.

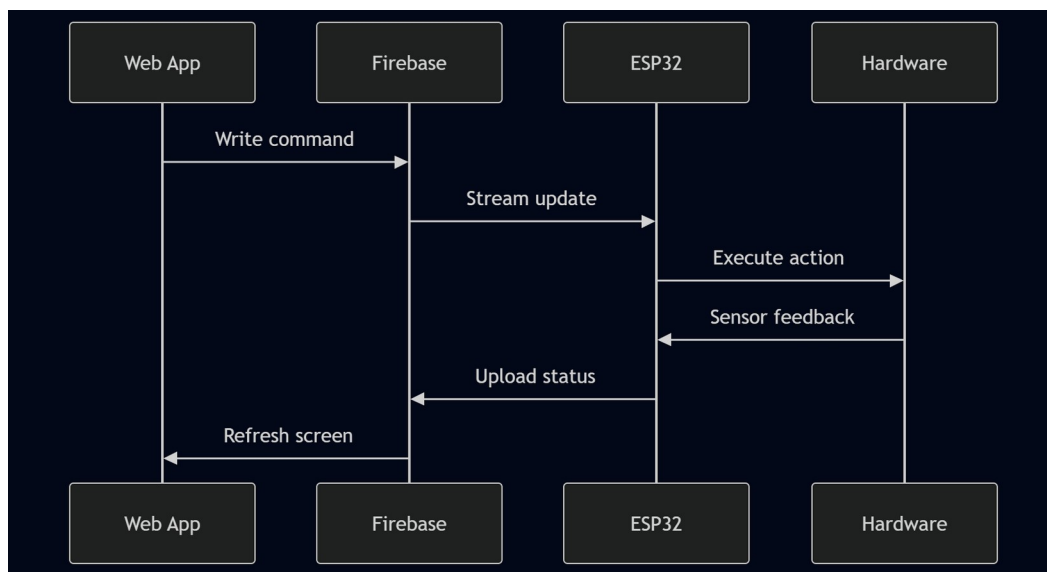
The combined manuscript brings together the earlier review work into one coherent document that can be used for submission, presentation, and final evaluation. The most important future improvement area is power optimization, especially if the design is later moved toward a battery-first commercial version.

Diagrams

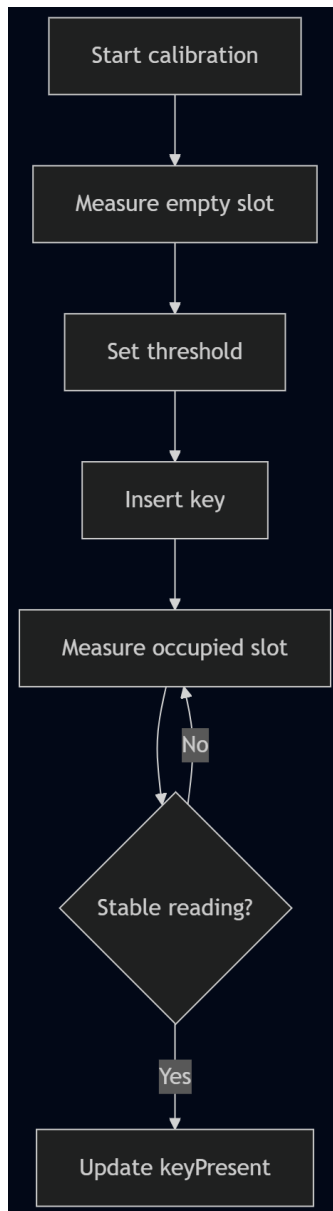
System flow



Timing



Calibration



Github Link: - <https://github.com/venkatasriramt-spec/smart-lockbox>

Video Link: - https://drive.google.com/file/d/1E9RE_bLEjM_qFtxsSgggLJNm_ikXJtsr/view?usp=sharing